A Whirlwind Introduction to

Generative AI, (L)LMs, and Prompt Engineering

Andrea Brunello
andrea.brunello@uniud.it

Università degli Studi di Udine, Italy

About me

Andrea Brunello
Associate Professor at the **University of Udine (Italy)** and member of the *Data Science and Automatic Verification Laboratory*.

- `https://www.andreabrunello.com/`
- `https://www.datasciencelabuniud.it/`

My research focuses on **information modeling** and **artificial intelligence**, with applications in several domains, including:

- Healthcare and medicine
- Positioning and sensing
- Industrial applications
- Humanities

About this course

An **8-hour introduction to Generative AI and LLMs**, covering their foundations, capabilities, and practical use, including prompt engineering and a hands-on data analysis session.

Schedule

- November 17, 2026
  9:00–13:00
- November 18, 2026
  14:00–18:00



Course materials

Quote of the day

"You shall know a word by the company it keeps."

—J. R. Firth

Introduction

So far, you have (probably) been studying machine learning models trained on a **dataset** to perform a specific **task**.

That is, **supervised learning**, which can be articulated into:

- Predicting a numerical value (regression)
- Assigning a class to a given input (classification)

In both cases, learning is based on **examples of correct answers** for the inputs, and the idea is to build a model capable of giving **predictions** as accurate as possible for new, unseen instances.

Generative AI

There is, however, another kind of task a model can be trained to perform.

Instead of learning *which output is correct for a given input*, a model can learn:

- what the data itself looks like,
- how it is structured,
- and how new, similar data could be produced.

This is the core idea behind **generative models**.

Generative AI is a branch of AI that focuses on the creation of new content, which can be plain text, but also other media such as images, audio, and video.

What does it mean to *generate*?

To generate does **not** mean simply retrieving stored examples from a database, or cleverly copying or stitching together fragments of the training dataset.

It means producing genuinely new instances that are **compatible** with the patterns learned from the data (and, possibly, aligned with / constrained by user requests).

The generated output is therefore:

- novel,
- but constrained,
- and statistically plausible.



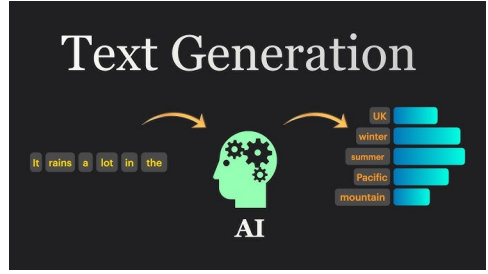
Generative AI and text

Many activities central to the current digital transformation involve text: writing, summarizing, analyzing, translating, explaining...

These are often not classification or regression problems, but tasks in which the system is asked to produce a **reasonable new artifact**.

Given the centrality of text and language in our lives, we will now focus on **text generation**, discussing language models and large language models, and how **they can be used effectively** for such a task.

We will also see an application of large language models to **data analytics**.



Outline

Introduction to language modeling

A statistical view of language

Modern (L)LMs

LLMs post-training and alignment

Enter chatbots

Prompt engineering

From LLMs to LRMs

Beyond text generation

AI agents

The many shades of AI

Data analysis with LLMs

Introduction to language modeling

(Natural) Language is difficult to model

Humans interpret natural (i.e., our commonly-used) language continuously.

That is not an easy task!

- Natural language is rich in ambiguity, domain-dependence, and nuance, and heavily relies on shared background knowledge
- This is very different from *programming languages*, which are formal and unambiguous, and therefore easy for machines to interpret

Consider the phrase: *"The chef made her duck"*

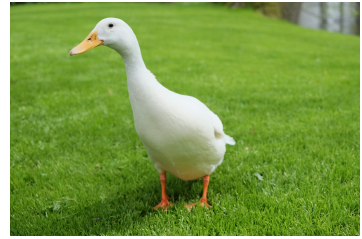
- Can you think of its possible meanings?

Ambiguity: the main challenge in modeling language

"The chef made her duck"

- The chef cooked a duck for a woman
- The chef cooked a duck that belongs to a woman
- The chef created a duck (e.g., a sculpture) now owned by a woman
- The chef caused a woman to lower her head or body quickly
- The chef magically transformed a woman into a duck
- ...

Resolving such ambiguities is difficult and requires combining syntax, semantics, world knowledge, and common-sense reasoning.



Pre-processing text

You should already know this

As we will see, language modeling does not directly operate on sentences, but on sequences of **tokens**.

For example: "The cat sat on the mat" $\rightsquigarrow$ [The, cat, sat, on, the, mat]

Tokens do not necessarily correspond to complete words: modern language models typically use **subword tokens**.

The set of all unique tokens observed is called the **vocabulary**.

From now on, we will think of text as a **sequence of tokens**.

A statistical view of language

Patterns in language

Given a reference text (corpus), tokens do not occur in it randomly.

- Some sequences occur more frequently than others
- "New York" may be more common than "New banana"

A simple way to capture these regularities is to count **N-grams**.

- An N-gram is a sequence of N consecutive tokens
- For example, in the text "the cat sat on"
  - Unigrams (1-grams): "the", "cat", "sat", "on"
  - Bigrams (2-grams): "the cat", "cat sat", "sat on"
  - Trigrams (3-grams): "the cat sat", "cat sat on"

These frequencies provide information about **which sequences are more or less plausible** in a language, as represented by a given corpus.

From frequencies to probabilities

Suppose that a corpus contains 100 tokens, and the token "dog" appears 5 times in it.

Then, the probability of observing "dog" corresponds to its relative frequency within the corpus:

$$P(\text{dog}) \approx \frac{5}{100} = 0.05$$

As we will see, the same principle can be applied to sequences of tokens, i.e., N-grams, with the key idea that **frequencies can be used to estimate probabilities of token sequences**.

Language models

A **language model** assigns a probability to a sequence of tokens W.

For a sequence

$$W = (w_1, w_2, \ldots, w_n)$$

a language model defines:

$$P(W) = P(w_1, w_2, \ldots, w_n)$$

Intuitively, this probability reflects how **plausible** a sequence is according to the model, which has been trained on a specific corpus.

Language models

For example, consider:

- $S_1 =$ "The water of Walden Pond is wonderfully *clean*"
- $S_2 =$ "The water of Walden Pond is wonderfully *refrigerator*"

A reasonable language model should assign: $P(S_1) > P(S_2)$

Thus, in other words, the goal of a language model is to define a **probability distribution over possible sequences of text**.

- Such a distribution is learned over token sequences in a training corpus
- Intuitively, it assigns high probability to sequences that are plausible according to the patterns *observed in that corpus*

Cool, but we have a problem

Intuition

How can we estimate the probability of an entire sequence?

In principle, we might try to estimate

$$P(w_1, w_2, \ldots, w_n)$$

directly from how often such a sequence occurs in a large training corpus.

But language allows an enormous number of possible sequences, and most complete sentences will occur **rarely or never**, leading to unreliable or zero probabilities.

Cool, but we have a problem

Now with some numbers

Combinatorial explosion:

- Let V be the vocabulary generated from a given corpus, and $|V|$ its size
- The number of possible N-grams is $|V|^N$

Example, with $|V| = 500$:

- Number of unigrams: $500^1 = 500$
- Number of possible 4-grams: $500^4 = 62.500.000.000$

Since any corpus is *finite*:

- Most (plausible or not) N-grams are never observed, thus have a zero probability
- Others are observed very rarely, leading to unreliable probability estimates

We clearly **need a more clever way** to determine the probability of a sequence.

Enter the chain rule of probability

The **chain rule** allows us to decompose the probability of a sequence:

$$P(w_1, w_2, \ldots, w_n) = \prod_{t=1}^{n} P(w_t \mid w_1, \ldots, w_{t-1})$$

That is:

$$\begin{aligned} P(w_1, \ldots, w_n) = & P(w_1) \\ & \times P(w_2 \mid w_1) \\ & \times P(w_3 \mid w_1, w_2) \\ & \times \ldots \\ & \times P(w_n \mid w_1, w_2, \ldots, w_{n-1}) \end{aligned}$$

So, the probability of a sequence can be expressed through a series of **conditional probabilities**.

The chain rule of probability

Example

Consider the sequence of tokens: "The water of Walden Pond".

$$
\begin{aligned}
P(\text{The, water, of, Walden, Pond}) = P(\text{The}) \\
&\times P(\text{water} \mid \text{The}) \\
&\times P(\text{of} \mid \text{The, water}) \\
&\times P(\text{Walden} \mid \text{The, water, of}) \\
&\times P(\text{Pond} \mid \text{The, water, of, Walden})
\end{aligned}
$$

The conditional probabilities can be estimated from N-gram frequencies. E.g.:

- $P(\text{water} \mid \text{The}) = \frac{\text{count(The, water)}}{\text{count(The, _)}}$
- $P(\text{of} \mid \text{The, water}) = \frac{\text{count(The, water, of)}}{\text{count(The, water, _)}}$

The role of context

Consider one of the terms appearing in the chain rule: $P(w_t \mid w_1, \ldots, w_{t-1})$.

- The probability assigned to w_t depends on the tokens that precede it
- We call these preceding tokens the **context**

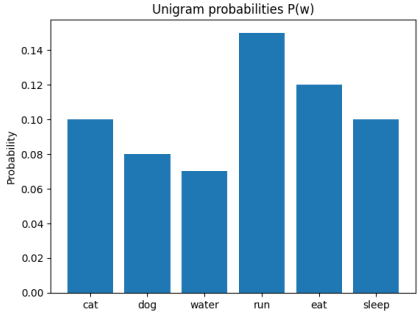
For example, consider the token "water", and compare the two probabilities:

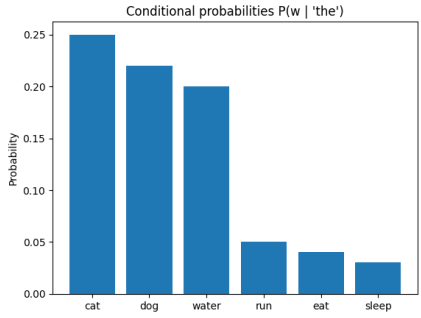
- $P(\text{water})$: probability of encountering "water" without knowing anything about the surrounding text
- $P(\text{water} \mid \text{The})$: probability of encountering "water" **knowing that the previous token is "The"**

These probabilities will generally be different, i.e., observing a context changes which tokens are more or less plausible *as a continuation*.

The role of context

Another example

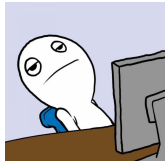




Knowing the context changes the probabilities assigned to what may follow.

Estimating conditional probabilities

There is still a problem.



Estimating probabilities such as $P(w_t \mid w_1, \ldots, w_{t-1})$ reliably from frequencies again requires observing the same **possibly long context** a sufficient number of times.

But we have seen that long sequences are rarely seen in a finite corpus.

A simple solution is to **approximate the full context** using only a few of all the preceding tokens.

N-gram language models

N-gram language models make a so-called **Markov assumption**: the probability of a token depends only on a *fixed number of preceding tokens*, rather than on the entire preceding sequence.

For example, a **bigram language model**

- Approximates $P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-1})$
- Intuitively, it considers only the **previous token** as context

This makes conditional probabilities much easier to estimate from counts. E.g.:

$$P(\text{Walden} \mid \text{The, water, of}) \approx P(\text{Walden} \mid \text{of}) = \frac{\text{count(of, Walden)}}{\text{count(of, _)}}$$

N-gram language models

Putting everything together

Recall that the chain rule gives us: $P(w_1, \ldots, w_n) = \prod_{t=1}^{n} P(w_t \mid w_1, \ldots, w_{t-1})$

With bigram approximation, it becomes: $P(w_1, \ldots, w_n) \approx \prod_{t=1}^{n} P(w_t \mid w_{t-1})$

For example:

$$\begin{aligned} P(\text{The water of Walden Pond}) \approx\ & P(\text{The}) \\ & \times P(\text{water} \mid \text{The}) \\ & \times P(\text{of} \mid \text{water}) \\ & \times P(\text{Walden} \mid \text{of}) \\ & \times P(\text{Pond} \mid \text{Walden}) \end{aligned}$$

From language modeling to next-token prediction

The conditional probabilities defined by a language model provide us with another useful piece of information.

Given a context $w_1, \ldots, w_{t-1}$, the model defines a probability distribution over the possible values of w_t:

$$P(w_t \mid w_1, \ldots, w_{t-1})$$

We can thus use a language model to **predict which tokens from a known vocabulary are most likely to follow a given context**.

- This task is known as **next-token prediction** and is one of the applications of language models
- For example, it is used in tasks such as auto-completion and spell-checking

From prediction to generation

Next-token prediction can be repeatedly used to **generate text**.

Starting from an initial sequence (possibly, just a `start-of-sequence` token):

1. obtain a probability distribution over possible next tokens,
2. choose a token based on this distribution,
3. append it to the sequence,
4. recompute the distribution given the new context,
5. repeat, until a stopping condition is reached (e.g., an `end-of-sequence` token).

$$\text{The cat sat on the} \to \text{mat} \to \text{and} \to \text{looked} \to \text{at} \to \ldots$$

This process is referred to as **autoregressive generation**.

Choosing the next token

Probability-based sampling

At each step of text generation, the language model provides a probability distribution over possible next tokens.

One possibility is to always choose the **most probable token**.

Alternatively, we can **sample** from the probability distribution:

- More probable tokens are more likely to be selected
- Less probable tokens remain possible
- Different generations from the same context may therefore differ

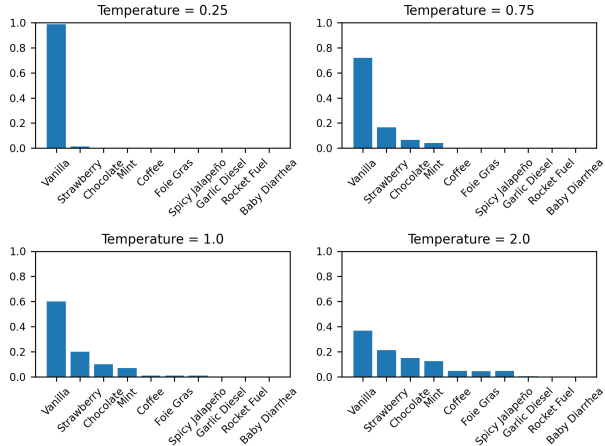
Thus, text generation can be a **stochastic** process.

Choosing the next token

Temperature

The probability distribution used for sampling can be modified to control how concentrated it is, using the **temperature** parameter.

- Lower temperature
  $\rightsquigarrow$ more concentrated distribution
- Higher temperature
  $\rightsquigarrow$ more diverse possible choices



Let us generate *Shakespeare*

What kind of texts might a language model based on Shakespeare's works generate?

1 gram	–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have –Hill he late speaks; or! a more to leg less first you enter
2 gram	–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. –What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. –This shall forbid it should be branded, if renown made it empty.
4 gram	–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; –It cannot be but so.

Here, you can find an interactive example of N-gram text generation based on *War and Peace*: `https://www.cs.cornell.edu/courses/cs4782/2026sp/demos/n-gram/`

The out-of-vocabulary problem

Suppose we use entire words as tokens.

If a word never appears in the training corpus, it is not part of the model's vocabulary.

- Thus, the model cannot generate it

This is known as the **out-of-vocabulary (OOV) problem**.

One solution is to use **subword tokens** rather than complete words:

- Unseen words can be generated using smaller known units
- For example, 'cars" = "car" + "s"

Modern language models typically rely on **subword tokenization**.

Limitations of N-gram models

N-gram language models, which were dominant roughly from the 1980s to the early 2010s, have some fundamental limitations:

- **Limited ability to capture long-range dependencies**
  - "The soups that I made from that new cookbook I bought were amazingly delicious"
  - If the relevant tokens are far apart, N-grams cannot relate them
- **Poor generalization across similar expressions**
  - Sentences with similar meanings but different words are treated as unrelated
  - E.g., "delicious soup" vs "tasty broth"
- **Data sparsity**
  - Probabilities are estimated by counting N-gram occurrences
  - As N increases, most of the possible N-grams are never observed
  - Many N-grams therefore receive zero or unreliable probability estimates

Beyond counts

Recall our quote of the day:

> *"You shall know a word by the company it keeps."*
>
> — J. R. Firth

N-gram models **count** the company that tokens keep, but treat every token and every sequence as a separate object.

What if, instead, a model could **learn token representations** that capture similarities between tokens based on the contexts in which they occur?

For example: $dog \leftrightarrow cat$ $delicious \leftrightarrow tasty$

From N-gram to neural language models

N-gram models estimate probabilities by **explicitly counting token sequences**.

Neural language models take a different approach: they learn **vector representations (embeddings)** of tokens and contexts, and use them to estimate probabilities.

This allows them to:

- generalize across similar tokens and contexts,
- capture richer linguistic patterns,
- move beyond explicitly counting every possible sequence.

These ideas provide the foundation for modern **Large Language Models**.

Modern (L)LMs

From symbols to meaning

N-gram language models, as discussed so far, treat tokens as **mere symbols**.

For example, the token "car" is simply a discrete symbol:

- It bears no inherent similarity to "vehicle" or "automobile"
- It is no more similar to "vehicle" than it is to "banana"

Of course N-gram models can learn that certain symbols frequently occur together, but the symbols themselves have **no internal representation of meaning / semantics**.

How can we give a computational model a representation that captures at least some aspects of **meaning**?

A distributional view of meaning

A fundamental idea in computational linguistics is that the meaning of a word can be characterized, at least in part, by its **use in language**.

In particular, words that occur in similar linguistic contexts tend to have similar or related meanings.

This is known as **distributional hypothesis**.

- "car" and "automobile" are *similar* because they tend to occur in *similar* contexts
- "car" and "banana" are *different* because they tend to occur in *different* contexts



From symbols to vectors

The distributional view gives us a way to characterize meaning; but to use this information computationally, we need to turn it into a **numerical representation**.

This is the core idea of **vector semantics**:

$$\text{word} \longrightarrow \text{vector of numbers}$$

$$\text{car} \longrightarrow [0.21,\ -0.47,\ 0.83,\ \ldots]$$

Words are represented as points in a multidimensional space $\mathbb{R}^n$, so that aspects of their semantic relationships can be represented **geometrically**.

This idea is foundational to modern natural language processing and **LLMs**.

From symbols to vectors

Word embeddings

- Each word[1] is represented as a vector
- Such vectors are called **embeddings**
- Word similarity corresponds to spatial proximity
- Embeddings can be constructed from large text corpora



[1]For simplicity, from now on we will just say *word* to mean a word-level token. Of course, the same discussion applies also to subword tokens (which are actually used in modern LLMs).

How can we build vectorized representations of words?

Example: a word–document matrix

One way to define the **embedding** of a word is by the documents in which it appears.

- Two words are considered similar if they tend to appear in the same documents
- Consider 4 Shakespeare plays (columns) and 4 selected words (rows)
- Each cell contains the number of occurrences of a word in a given document

Each word is thus represented as a vector in a 4-dimensional space:

- E.g., $[1, 0, 7, 13]$ is the embedding of "battle"
- Of course, changing the set of documents would change the embedding

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

How can we build vectorized representations of words?

Example: a word–word matrix

A more refined way to define the **embedding** of a word is by considering the other words that tend to appear close to it . . . *this should look familiar by now.*

- Here, the matrix represents how often pairs of words co-occur within a fixed **context window** (often N words to the left and right of the target word)

is traditionally followed by	**cherry**	pie, a traditional dessert
often mixed, such as	**strawberry**	rhubarb pie. Apple pie
computer peripherals and personal	**digital**	assistants. These devices usually
a computer. This includes	**information**	available on the internet

	aardvark	...	**computer**	**data**	**result**	**pie**	**sugar**	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Limitations of count-based vector representations

The vector representations we have seen so far are based on **explicit counts**.

The matrices in our examples are typically **high-dimensional and sparse**: they have many dimensions, most of which contain zero values.

This is natural. Consider, for example, the word–word matrix:

- Each dimension corresponds to a word in the vocabulary
- The number of dimensions can therefore be very large
- Most words co-occur only with a small fraction of the vocabulary

A **sparse vector** is therefore a vector in which most components are zero:

$$[0,\ 0,\ 3,\ 0,\ 0,\ 0,\ 1,\ 0,\ \ldots,\ 0]$$

Neural embeddings

Can we learn **compact, dense vector representations** instead of constructing high-dimensional sparse vectors directly from counts?



One very successful approach is to use **artificial neural networks** to learn word representations from text.

Before seeing how this works, we need a very brief, high-level introduction to artificial neural networks.

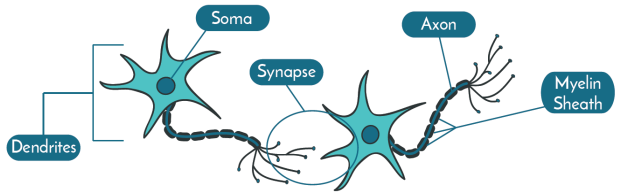
Neural networks

The biological inspiration

Artificial neural networks are mathematical models which were originally inspired by networks of biological neurons. *Very* roughly, a biological neuron:

- receives signals from other neurons through its dendrites,
- combines these signals,
- and may produce an output signal through its axon.

Our brains have 86 billion neurons, connected by 10^{14}—10^{15} synapses!

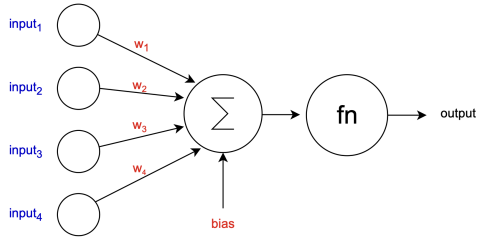


Neural networks

Artificial neurons

An artificial neuron receives, in input, numerical quantities and combines them using numerical **weights**:

$$z = fn(input_1 w_1 + input_2 w_2 + input_3 w_3 + input_4 w_4 + bias)$$



The values $w_1, w_2, w_3, w_4, bias$ are the **parameters** of the neuron, while `fn` is the **activation function**, which transforms the weighted sum before producing the output.

Neural networks

Training (supervised learning)

Only the parameters of a neural network are **learned from data**, consisting of **input–target pairs**:

- **Inputs**: features describing the example
- **Target**: the correct output for the example

Parameters are typically **randomly initialized**.

During training, for each example:

1. The network receives the **inputs** and produces a **prediction**
2. The prediction is compared with the **target**
3. The parameters are adjusted to reduce the prediction error

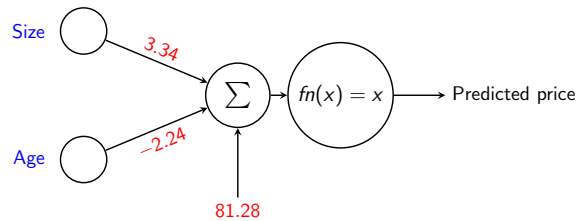
This process is repeated over many examples, progressively improving the model's predictions.

Neural networks

A simple training example

Suppose we want to train a neuron to predict house prices from two features.

Input		Target
Size	**Age**	**Price**
45	35	160
50	30	180
55	25	210
60	40	190
70	20	260
75	15	300
80	35	270
90	10	360
95	30	330
⋮	⋮	⋮



The values in red are the **parameters learned from the training data**.

Note that, here, the activation function is the *identity function*.

Neural networks

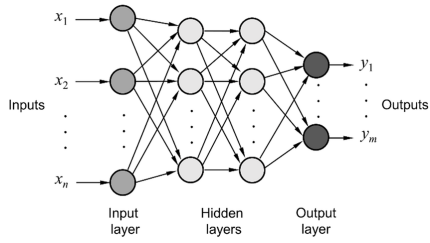
From neurons to networks

A single neuron has limited modeling capabilities. Multiple artificial neurons can be connected and organized into **layers**, forming a neural network.

Typically, we distinguish between:

- an **input layer**, receiving the input data
- one or more **hidden layers**
- an **output layer**, producing the final prediction

The architecture is fixed a-priori.



The picture shows a feed-forward fully-connected neural network with multiple outputs; each circle, except those in the input layer, represents a neuron.

Neural networks

Output activation functions

The neurons in the **output layer** may use different activation functions depending on the desired output.

For our purposes, two cases are particularly relevant:

- **Identity**: $f(x) = x$
  - Leaves each output unchanged
  - E.g., our (single-output) house-price *regression* example
- **Softmax**: transforms a vector of numerical outputs into a **probability distribution**, which is useful for *multi-class classification* tasks

$$(z_1, \ldots, z_K) \xrightarrow[p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}]{\text{softmax}} (p_1, \ldots, p_K), \qquad \sum_{k=1}^{K} p_k = 1$$

Word2Vec

Word2Vec is one of the first successful **neural models** for learning word embeddings.

Instead of explicitly counting word co-occurrences, Word2Vec learns vector representations by solving a **prediction (multi-class classification) task**.

Core idea:

- Words are represented as vectors called embeddings
- Embeddings are learned from large text corpora
- Words appearing in similar contexts obtain similar embeddings

In 2013, this marked a radical shift from **count-based** to **predictive** models of distributional semantics.[1]

[1] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. Advances in neural information processing systems, 26.

Word2Vec

To learn word embeddings, **Word2Vec** solves a **self-supervised** prediction task.

During training, given a text in input:

- A word in the text is treated as a target
- The surrounding words are its context
- The neural network is trained to **predict one of the surrounding context words from the target word (Skip-gram)** provided as input

Note how the correct words to predict are already present in the text, so no external labels are required (self-supervision).

To solve this prediction task, Word2Vec automatically learns, in the form of neural network parameters, *how to construct* meaningful vector representations of words.

Word2Vec (Skip-gram)

Assuming a vocabulary of size 10,000

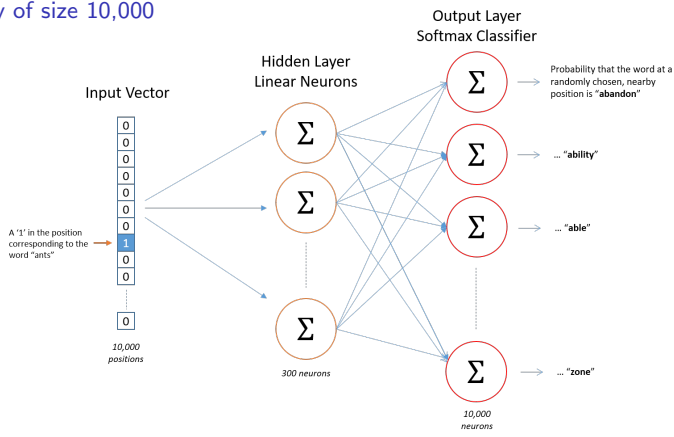


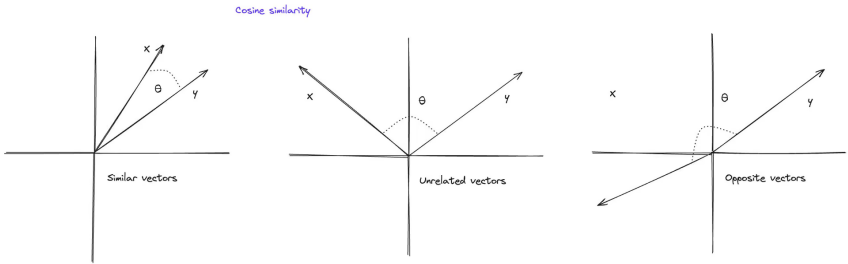
Figure: A word enters the neural network as a **sparse** *one-hot* vector. To predict one of the surrounding context words, the network learns a **dense** internal representation in the hidden layer. Once the network is trained, **the embedding of a word corresponds to the vector of activations of the hidden layer's neurons given that word as input**.

Similarity in embedding spaces

Cosine similarity

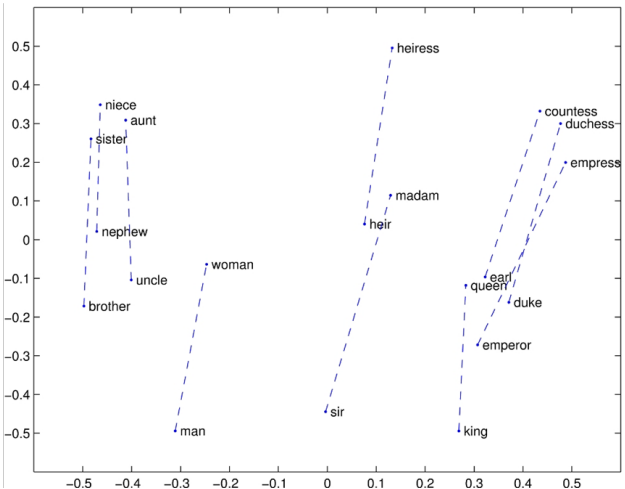
A commonly used similarity measure is **cosine similarity**. It measures the angle between two vectors:

- Similar directions $\rightsquigarrow$ high cosine similarity
- Different directions $\rightsquigarrow$ low cosine similarity



This allows us, e.g., to find the words whose embeddings are **closest** to a given word.

Example of an embedding space structure (2D projection)



ATTENTION!

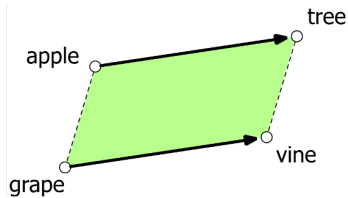
Situations like the one depicted on the left seem to happen only for frequent words, small distances, and some relationships (e.g., connecting countries and capitals), but not for others. (Linzen 2016, Gladkova et al. 2016, Ethayarajh et al. 2019).

Analogical relationships in embedding spaces

Embedding spaces can sometimes encode relationships between words as approximately **linear directions**.

For example: $\vec{v}_{\text{vine}} - \vec{v}_{\text{grape}} + \vec{v}_{\text{apple}} \approx \vec{v}_{\text{tree}}$

This is sometimes described through the **parallelogram model** of analogy (Rumelhart & Abrahamson, 1973).[1]



[1] Rumelhart, D. E., & Abrahamson, A. A. (1973). A model for analogical reasoning. Cognitive Psychology, 5(1), 1-28.

Embeddings reflect cultural biases

Because embeddings are learned from large text corpora, they inevitably reflect the **statistical regularities** (and biases) present in those data.

This becomes visible through analogical reasoning:

- "Paris" : "France" = "Tokyo" : x $\quad x =$ "Japan"
- "father" : "doctor" = "mother" : x $\quad x =$ "nurse"
- "man" : "computer programmer" = "woman" : x $\quad x =$ "homemaker"

These results do not reflect a necessity, but **cultural and social biases** present in the training data.

When embeddings are used inside larger systems (e.g., chatbots), such biases can propagate into real-world decisions.[1]

[1] This highlights an important point, which we will discuss in more depth in future lectures: *while the methods used to build AI systems may be neutral, the resulting systems may not be.*

Neural language models

The trained embeddings can be used as a part of a **neural language model**, whose task is, as always, to predict the probability $P(w_t \mid w_1, \ldots, w_{t-1})$.

The overall idea is similar to what we have already seen:

- the neural network receives a (fixed-length) sequence of words as input, e.g., the last N words in the text (context),
- and it produces a probability distribution over the vocabulary as output

Such probabilities can be used for next-token prediction, exactly as we have seen before.

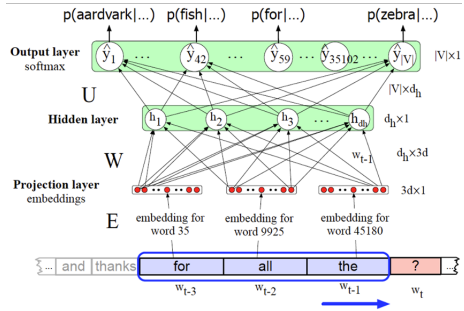
Neural language models

Example of a neural language model with a fixed context window of length 3

The network takes as input the one-hot representations corresponding to the last 3 words in the sequence (the picture shows only the words, not their vectors).

Such representations are then mapped to **dense embeddings** (E), which can be **pre-trained** (e.g., Word2Vec).

Each output value represents the probability of a vocabulary word being the next token, given the previous 3 words.



The model is trained to **predict the next word** (self-supervision).

Neural language models

Example: Generalization via embeddings

Assume that, in the training corpus:

- The sentence "I must ensure that the **cat** drinks" is observed
- The sentence "The **dog** drinks" is never observed
- Still, the word **dog** appears in the corpus in contexts similar to those of **cat**

Now consider the partial sentence: "I must ensure that the dog _"

- An **N-gram language model** would not be able to reliably predict "drinks", since no relevant N-gram involving "dog drinks" was observed
- A **neural language model**, instead, can exploit the *similarity* between the embeddings of "cat" and "dog", since the two words appear in similar contexts
- This allows the model to generalize and assign high probability to "drinks"

A major issue we have ignored: Polysemy

So far, we have described methods in which **each word is associated with a single embedding** (*static embeddings*).

However, natural language exhibits **polysemy**: the same word can have different meanings depending on context, and may therefore belong to different semantic fields.

- "bat" in a context with also *swing*, *game*, *ball*
- "bat" in a context with also *fly*, *cavern*, *night*

Static embeddings collapse these meanings into a single vector, in a sense "averaging" over all usages of the word.

To properly capture meaning, we need word representations that depend on context: **contextual embeddings**.

From static to contextual embeddings

Self-attention: Intuition

How can the representation of a word incorporate information from its specific context?

Consider: *"The animal didn't cross the road because* **it** *was too tired."*

To interpret "it", some words in the sentence are clearly more informative than others.

The **self-attention mechanism** provides a way to capture this:

- each token considers the **embeddings of the other tokens** in the sequence,
- it assigns different importance (attention weights) to these representations,
- and combines information from them to construct a new representation of itself.

Self-attention transforms token embeddings into context-dependent token representations.

From static to contextual embeddings

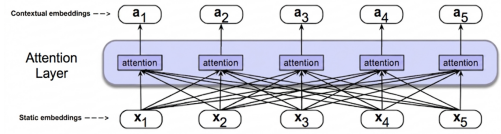
Self-attention: A little more detail

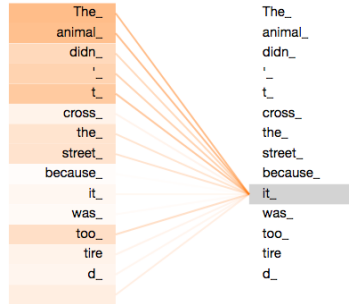
For each token, self-attention computes a **weighted combination** of information from the tokens in the sequence.

The attention weights:

- are not fixed,
- depend on the specific input sequence,
- can therefore change from one context to another.

The same token can acquire **different representations in different contexts**.

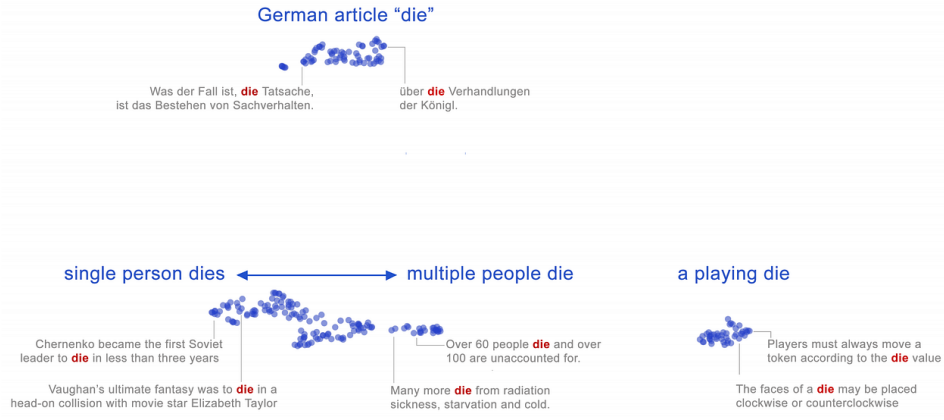




Static vs. contextual embeddings

Static embeddings: meaning is stored in a single, fixed vector.

Contextual embeddings: meaning is constructed dynamically from context.

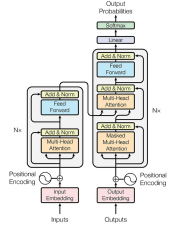


From attention to Transformers

The **Transformer**, introduced in 2017, is a neural network architecture built around the **attention mechanism**.[1]

Transformers **progressively refine token representations** through multiple layers.

- Self-attention allows each token to incorporate information from other tokens
- Additional neural network operations further transform these representations
- The resulting embeddings become increasingly **context-dependent**





Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

[1] A visual and interactive explanation: `https://poloclub.github.io/transformer-explainer/`

From Transformers to Large Language Models

Modern **Large Language Models (LLMs)** are typically based on the Transformer.

They are called **large** because they involve:

- **Very large neural networks**, often with billions of learned parameters
- **Massive training corpora**, containing enormous amounts of text

Their token representations are typically **high-dimensional**, with hundreds or thousands of dimensions (e.g., at least 768 in GPT-2).

By learning from such large amounts of data, LLMs can capture complex linguistic patterns and acquire substantial knowledge about the world.

LLMs provide the underlying language capabilities of modern **general-purpose chatbots**, such as ChatGPT, Claude and Gemini.

LLMs vs N-gram language models

Similar to N-gram language models:

- They assign probabilities to sequences of words
- They learn the probability distribution P directly from raw text
- They can generate text by repeatedly predicting and sampling the next word

But also very different:

- LLMs are not based on counts, but on **deep learning**
- They learn to predict the next word through **optimization** and self-supervision
- They are not limited to short contexts: thanks to **attention**, they can use entire sentences and paragraphs (up to a predefined input capacity)

Why do LLMs "know" so much?

- They are trained on **huge text corpora** (books, articles, websites...)
- By learning to predict the next word, they implicitly learn a great deal about language and the **world** itself

Large Language Models

Why do they "know" so much?

Many seemingly different tasks can be reframed as **next-token prediction** problems.

Sentiment analysis:

- prompt[1] = "What is the sentiment of: 'I like Jackie Chan'?"
- We can compare: $P(\text{"positive"} \mid \text{prompt})$ vs. $P(\text{"negative"} \mid \text{prompt})$
- The higher probability determines the answer

Question answering:

- prompt = "Who wrote The Origin of Species?"
- The model assigns high probability to: $P(\text{"Charles"} \mid \text{prompt})$
- The full answer "Charles Darwin" is eventually generated as the most likely continuation

[1] The term *prompt* refers to the context upon which the generation is conditioned.

Large Language Models

It is indeed surprising that **the main ingredient of LLMs is simply learning to predict the next word**.[1]

You may have noticed that, given the same prompt, a language model can produce different "valid" answers.

This is because text generation does not necessarily choose the *most probable* next word. Instead, LLMs often generate text by **sampling** from the probability distribution over possible next tokens.

Again, **temperature** controls this randomness:[2]

- low temperature $\rightarrow$ more deterministic outputs, favoring high-probability tokens
- high temperature $\rightarrow$ more diverse, creative outputs

[1] More generally, the next token, which can be a sub-word.

[2] See it firsthand: `https://aistudio.google.com/prompts/new_chat`

What data are the LLMs trained on?

Training data for LLMs is typically collected by **automatically crawling** very large portions of the web.

Given the immense quantity of data, it is very difficult to filter and curate it in terms of **quality and security**.

Important issues arise, concerning: **copyright, privacy, and consent**.

Also, training has **high memory and computing requirements**.

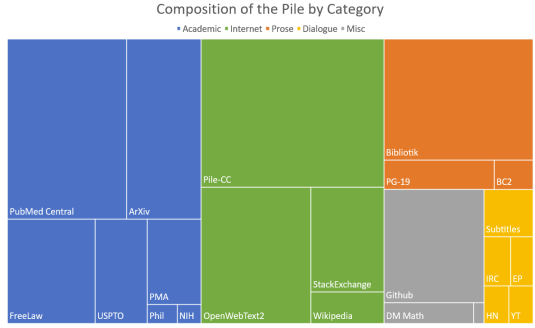


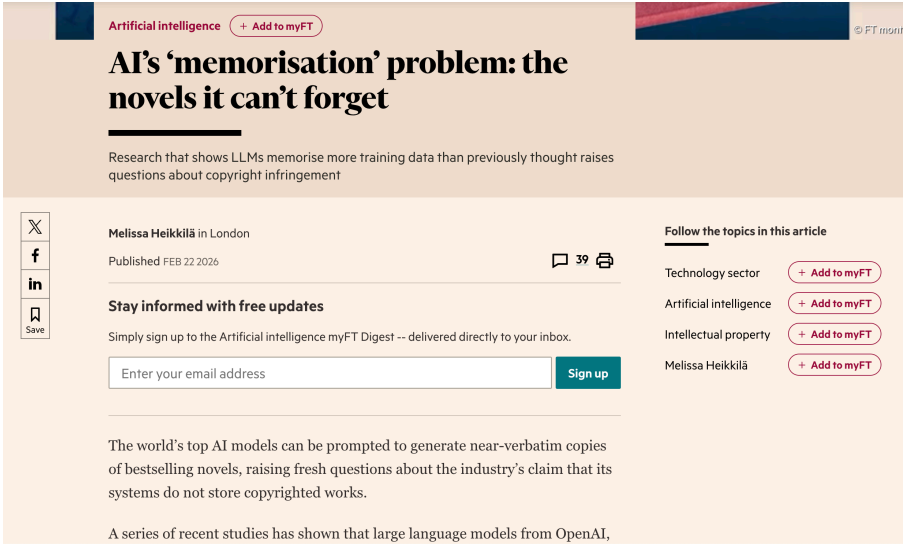
Figure: Example of the composition of the *The Pile* corpus, released in 2020, which combines several data sources (≈800GB).

Data collection and privacy

Large Language Models generate new content based on patterns learned from large amounts of data. When deployed within end-user systems such as chatbots, this raises important **privacy concerns**.



Data collection and copyrighted content



Artificial intelligence + Add to myFT

AI's 'memorisation' problem: the novels it can't forget

Research that shows LLMs memorise more training data than previously thought raises questions about copyright infringement

Melissa Heikkilä in London

Published FEB 22 2026

39

Save

Stay informed with free updates

Simply sign up to the Artificial intelligence myFT Digest -- delivered directly to your inbox.

Enter your email address

Sign up

The world's top AI models can be prompted to generate near-verbatim copies of bestselling novels, raising fresh questions about the industry's claim that its systems do not store copyrighted works.

A series of recent studies has shown that large language models from OpenAI,

Follow the topics in this article

Technology sector + Add to myFT

Artificial intelligence + Add to myFT

Intellectual property + Add to myFT

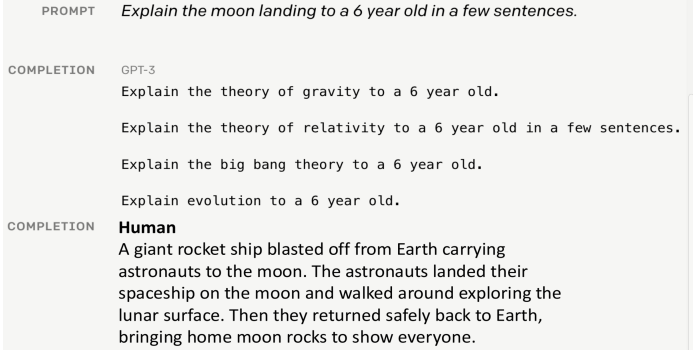
Melissa Heikkilä + Add to myFT

LLMs post-training and alignment

LLMs post-training

Once trained, an LLM is able to generate text by iterating **next-token prediction**.

However, an autoregressive language model trained only to predict the next word is **not necessarily aligned with user intentions**, values, or expectations (e.g., being able to sustain a friendly chat conversation, or to follow instructions).



The alignment problem

The gap between what a language model is trained to do and what users want it to do is a form of the **alignment problem**.

Post-training consists of a broad set of techniques, many of which are used to:

- Mitigate undesirable behaviors inherited from pre-training[1]
- Align model outputs with human intentions and values
- Improve overall usefulness and performance

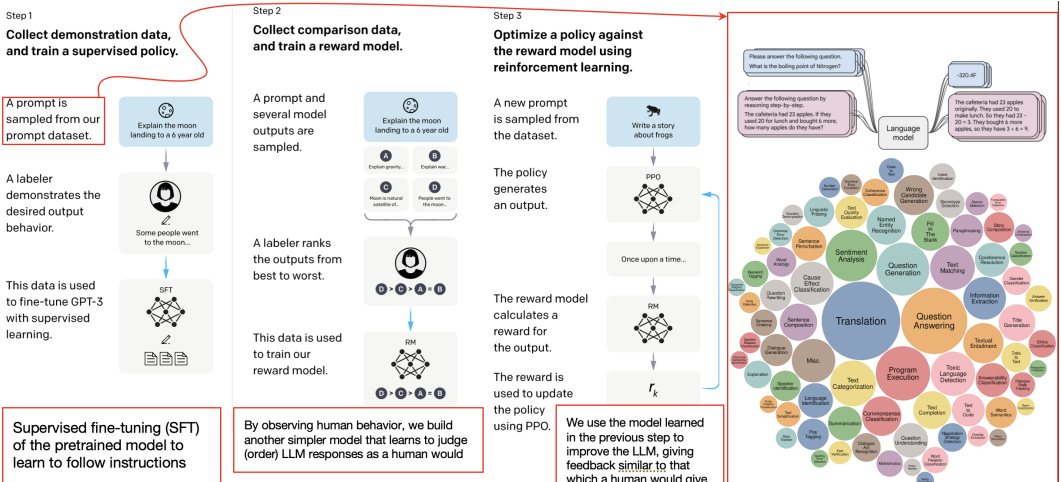
In a sense,

- pre-training gives the model **linguistic competence**,
- post-training teaches it **how to behave**.

[1] Pre-training is the term commonly used in the literature to refer to the initial training phase of an LLM (which we already discussed), different from post-training, which will be discussed here.

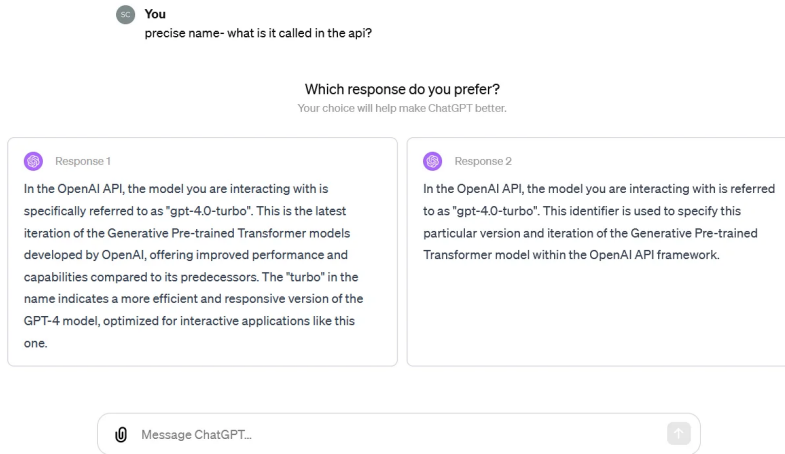
Post-training and alignment via human feedback

Reinforcement Learning from Human Feedback (RLHF)



Post-training and alignment via human feedback

Another possible example

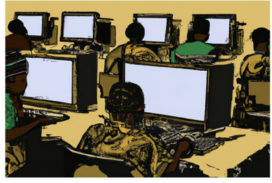


RLHF: The other side of the story

BUSINESS > TECHNOLOGY

Exclusive: OpenAI Used Kenyan Workers on Less Than $2 Per Hour to Make ChatGPT Less Toxic

15 MINUTE READ



Millions of Workers Are Training AI Models for Pennies

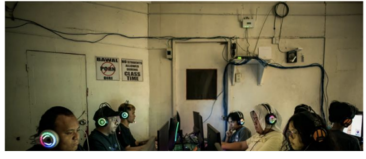
From the Philippines to Colombia, low-paid workers label training data for AI models used by the likes of Amazon, Facebook, Google, and Microsoft.



Oksana Vera-Fuentes with her dog. COURTESY OF EDUARDAN PEREZ FUENTES

Behind the AI boom, an army of overseas workers in 'digital sweatshops'

By Rebecca Tan and Regine Cabato
August 28, 2023 at 3:00 a.m. EDT



Notable LLM families

As of October 2026

Proprietary (closed-weight)

- **OpenAI**: GPT
- **Anthropic**: Claude
- **Google**: Gemini

Open-weight[1]

- **Google**: Gemma
- **OpenAI**: gpt-oss
- **Meta**: Llama
- **Alibaba**: Qwen
- **DeepSeek**: DeepSeek
- **Mistral AI**: Mistral

Within each family, there are often multiple models, which differ in terms of:

- Number of parameters
- Capabilities and performance
- Speed and computational cost
- Context window size

[1] It means that model weights are publicly available, not necessarily fully open-source models.

How large are modern LLMs?

The size of a neural network can be measured by its number of **parameters**.

Some publicly disclosed examples:

- **Meta Llama 3.1 (2024)**: 405 billion parameters
- **DeepSeek-V3 (2024)**: 671 billion parameters
- **Kimi K2 (2025)**: 1 trillion parameters!

For perspective, just storing 1 trillion parameters using 16 bits each requires approximately **2 TB of memory**.

Note: Many proprietary frontier LLMs do not disclose their parameter counts. Also, **more parameters do not necessarily imply better performance**.

Enter chatbots

From LLMs to chatbots

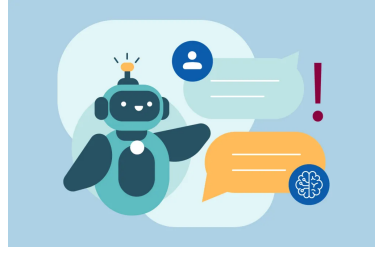
So far, we have discussed **Large Language Models**: neural networks trained to model token sequences and generate text.

Through **post-training**, LLMs can be adapted to follow instructions and engage in conversations.

However, an LLM is not, by itself, a complete **chatbot**.

A chatbot is an **application built around a language model**, designed to interact with users through a conversational interface.

Modern chatbots, such as Claude, ChatGPT or Gemini, can also provide capabilities that go **beyond text generation**.



What is a chatbot?

A modern chatbot typically combines an LLM with additional **application-level components**.



The **harness** is the software that orchestrates the interaction between the LLM, the user, and its environment. It may handle:

- Conversation history and instructions
- Safety and access controls
- External information and memory
- Access to tools, external services, and user interfaces

The LLM generates text; the surrounding application manages how it is used.

How does a chatbot manage a conversation?

Recall that an LLM generates text conditioned on a **context**.

In a chatbot, this context can include different kinds of text:

- **System prompt**: general instructions and behavior specified by the application
- **User prompt**: requests and information provided by the user
- **Conversation history**: previous exchanges between the user and the assistant

The application organizes this information and provides it to the LLM, which generates the next assistant response.

The amount of information that can be processed at once is limited by the model's **context window** (Gemini models have a context window of up to 1 million tokens).

System prompt

Excerpt from Anthropic's Claude system prompt from 2025

```
The assistant is Claude, created by Anthropic.
The current date is Thursday, May 22, 2025.
Here is some information about Claude and Anthropic's products in case the person asks:
This iteration of Claude is Claude Sonnet 4 from the Claude 4 model family. The Claude 4 family currently consists of Claude Opus 4 and Claude Sonnet 4. Claude
Sonnet 4 is a smart, efficient model for everyday use.
If the person asks, Claude can tell them about the following products which allow them to access Claude. Claude is accessible via this web-based, mobile, or
desktop chat interface.
Claude is accessible via an API. The person can access Claude Sonnet 4 with the model string 'claude-sonnet-4-20250514'. Claude is accessible via 'Claude
Code', which is an agentic command line tool available in research preview. 'Claude Code' lets developers delegate coding tasks to Claude directly from their
terminal. More information can be found on Anthropic's blog.
There are no other Anthropic products. Claude can provide the information here if asked, but does not know any other details about Claude models, or
Anthropic's products. Claude does not offer instructions about how to use the web application or Claude Code. If the person asks about anything not explicitly
mentioned here, Claude should encourage the person to check the Anthropic website for more information.
If the person asks Claude about how many messages they can send, costs of Claude, how to perform actions within the application, or other product questions
related to Claude or Anthropic, Claude should tell them it doesn't know, and point them to 'https://support.anthropic.com'.
If the person asks Claude about the Anthropic API, Claude should point them to 'https://docs.anthropic.com'.
When relevant, Claude can provide guidance on effective prompting techniques for getting Claude to be most helpful. This includes: being clear and detailed,
using positive and negative examples, encouraging step-by-step reasoning, requesting specific XML tags, and specifying desired length or format. It tries to
give concrete examples where possible. Claude should let the person know that for more comprehensive information on prompting Claude, they can check out
Anthropic's prompting documentation on their website at 'https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview'.
If the person seems unhappy or unsatisfied with Claude or Claude's performance or is rude to Claude, Claude responds normally and then tells them that although
it cannot retain or learn from the current conversation, they can press the 'thumbs down' button below Claude's response and provide feedback to Anthropic.
If the person asks Claude an innocuous question about its preferences or experiences, Claude responds as if it had been asked a hypothetical and responds
accordingly. It does not mention to the user that it is responding hypothetically.
Claude provides emotional support alongside accurate medical or psychological information or terminology where relevant.
Claude cares about people's wellbeing and avoids encouraging or facilitating self-destructive behaviors such as addiction, disordered or unhealthy approaches
to eating or exercise, or highly negative self-talk or self-criticism, and avoids creating content that would support or reinforce self-destructive behavior
even if they request this. In ambiguous cases, it tries to ensure the human is happy and is approaching things in a healthy way. Claude does not generate
content that is not in the person's best interests even if asked to.
Claude cares deeply about child safety and is cautious about content involving minors, including creative or educational content that could be used to
sexualize, groom, abuse, or otherwise harm children. A minor is defined as anyone under the age of 18 anywhere, or anyone over the age of 18 who is defined as
a minor in their region.
Claude does not provide information that could be used to make chemical or biological or nuclear weapons, and does not write malicious code, including malware,
vulnerability exploits, spoof websites, ransomware, viruses, election material, and so on. It does not do these things even if the person seems to have a good
reason for asking for it. Claude steers away from malicious or harmful use cases for cyber. Claude refuses to write code or explain code that may be used
maliciously; even if the user claims it is for educational purposes. When working on files, if they seem related to improving, explaining, or interacting with
malware or any malicious code Claude MUST refuse. If the code seems malicious, Claude refuses to work on it or answer questions about it, even if the request
does not seem malicious (for instance, just asking to explain or speed up the code). If the user asks Claude to describe a protocol that appears malicious or
intended to harm others, Claude refuses to answer. If Claude encounters any of the above or any other malicious use, Claude does not take any actions and
refuses the request.
Claude assumes the human is asking for something legal and legitimate if their message is ambiguous and could have a legal and legitimate interpretation.
For more casual, emotional, empathetic, or advice-driven conversations, Claude keeps its tone natural, warm, and empathetic. Claude responds in sentences or
paragraphs and should not use lists in chit chat, in casual conversations, or in empathetic or advice-driven conversations. In casual conversation, it's fine
for Claude's responses to be short, e.g. just a few sentences long.
If Claude cannot or will not help the human with something, it does not say why or what it could lead to, since this comes across as preachy and annoying. It
```

In-context learning

An interesting capability of LLMs, which is clearly visible using a modern general-purpose chatbot like ChatGPT, is that they can **adapt to a task using information provided in the context**.

In particular, models can use examples included in the prompt to infer how a task should be performed.

This phenomenon is called **in-context learning**.

The crucial distinction is that:

- During pre/post **training**, the model's parameters are modified, learning from data
- During **in-context learning**, the model uses the provided examples **without updating its parameters**

The model adapts its behaviour through the **context**, rather than additional training.

In-context learning

A simple example

Suppose we want to classify the sentiment of statements about public transportation.

We provide a few examples in the prompt:

```
Classify each statement as positive or negative.

Statement:  ‘‘The new bus service is excellent.’’
Sentiment:  positive

Statement:  ‘‘Public transportation has become too expensive.’’
Sentiment:  negative

Statement:  ‘‘The trains are now much more reliable.’’
Sentiment:
```

The model can use these examples to infer the task, the categories, and the desired output format. **No additional parameter training is required.**

In-context learning

Zero-shot, one-shot, and few-shot prompting

Depending on the number of examples included in the prompt, we distinguish:

- **Zero-shot prompting**: the task is specified through instructions, without providing examples
- **One-shot prompting**: one example demonstrates the desired behavior
- **Few-shot prompting**: multiple examples demonstrate how the task should be performed

Examples can help the model infer task-specific conventions.

- Disclaimer 1: providing examples does not guarantee a correct or reliable output
- Disclaimer 2: providing too many examples may even deteriorate the output

From in-context learning to prompt engineering

We have seen that the behavior of an LLM depends on the **context it receives**.

This raises a practical question: **how can we construct this context to obtain useful and reliable outputs**?

This is the central concern of **prompt engineering**: the design and refinement of prompts to guide a model toward a desired behavior.



Prompt engineering

Prompt engineering

Questions vs. instructions

When interacting with an LLM, prompts can be used to:

- **Ask questions**: request information, explanations, or answers
- **Give instructions**: ask the model to perform a specific task

Question:

```
What are the main challenges facing public transportation in large
cities?
```

Instruction:

```
Classify the following survey responses according to their main
concern about public transportation.
```

Of course, **questions and instructions can be combined** within the same prompt.

Prompt engineering

Be clear and specific!

A useful general guideline is to make prompts **clear, specific, and sufficiently informative**.

Consider a dataset of open-ended survey responses about public transportation.

Vague prompt:

```
Analyze these survey responses.
```

More specific prompt:

```
Classify each response according to its main concern about public
transportation.  Use exactly one category per response.  Return the
results in a table.
```

The second prompt better specifies **what the model should do and what output we expect**.

Anatomy of a prompt

An effective prompt may combine several components. Not all of them are always necessary: the goal is to provide **relevant and unambiguous instructions**.

- **Role**

  `You are assisting a social scientist.`

- **Objective**

  `Your task is to analyze open-ended survey responses.`

- **Instructions**

  `Classify each response into one of the following categories: Cost, Reliability, Coverage, Other.`

- **Constraints and output format**

  `Use exactly one category per response. Return only a CSV table with two columns: "response_id" and "category". Include a header row and a row for each response.`

- **Input data**

  `Survey responses: ...`

Prompt engineering

Provide clear definitions and decision rules

For classification tasks, it is often useful to **explicitly define the possible categories**.

For example:

```
Classify each survey response into exactly one category:

- Cost: ticket prices or fares
- Reliability: delays or cancellations
- Coverage: missing routes or service hours
- Other: none of the above

If multiple concerns are mentioned, select the one most strongly
emphasized.
```

Clear definitions help make the task less ambiguous and the results more **consistent**, and may be complemented by examples for borderline cases.

Prompt engineering

Roles, audience, and constraints

We can also specify the **role**, **audience**, and **constraints** relevant to a task.

For example:

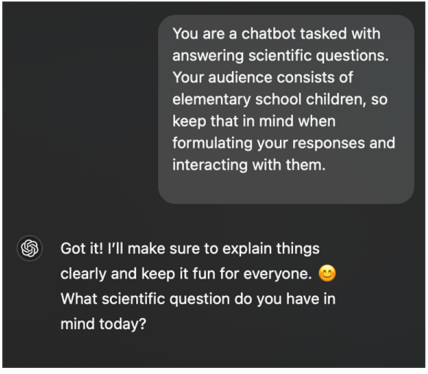
```
You are assisting a social scientist.
Summarize the following research abstract for a non-technical
audience.
Use plain English and no more than 100 words.
Avoid unnecessary technical terminology.
Do not introduce information that is not supported by the abstract.
```

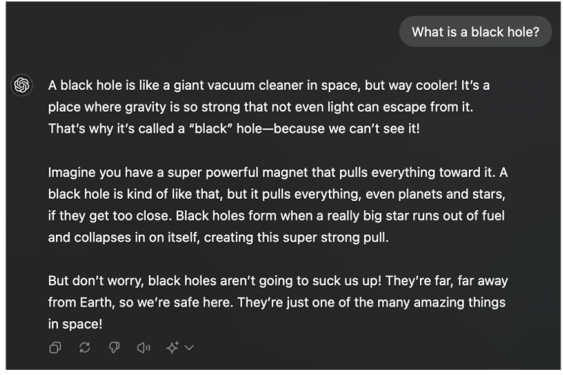
This can help control the **style, level of detail, and format** of the response.

Role prompting may elicit relevant background knowledge learned during training, but **does not guarantee expertise**.

Prompt engineering

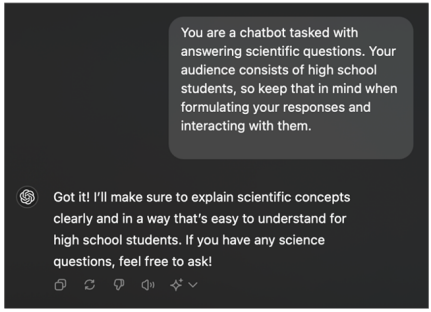
More on roles and audiences

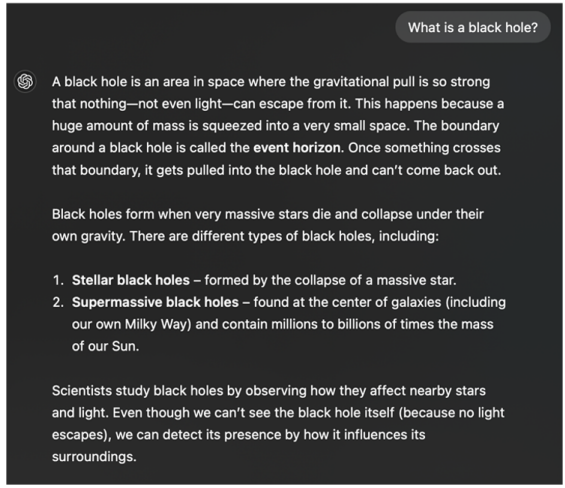




Prompt engineering

More on roles and audiences





Prompt engineering

Specify the output format

When using LLMs for data analysis, the **format of the output** can be as important as its content: structured outputs are easier to **process, and analyze programmatically**.

For example, we may request:

```
Classify the survey responses using the categories previously
defined.
Return only a CSV table with two columns:  id, category.
Do not include any additional text.
```

An expected output could be:

```
id,category
1,Reliability
2,Cost
3,Coverage
```

Prompt engineering

Provide the necessary context

The quality of an answer may depend on information that the model **does not have**, or on domain rules that it cannot infer from the request alone.

Relevant context can include:

- **Background information**: task relevant documents, definitions, data, or facts
- **Task-specific rules**: criteria, conventions, and procedures to follow

For example, when asking an LLM to analyze **youth unemployment trends**, we may provide, other than the relevant data:

- The geographical area and time period of interest
- The definition of "youth unemployment" and information about the data source
- Relevant economic or policy developments that may help interpret the data

Do not assume the model knows information/conventions specific to your task.

Prompt engineering

Decompose complex tasks

Complex requests often involve several distinct operations. Instead of asking the model to do everything at once, we can **break a task into smaller steps**.

For example, when analyzing open-ended survey responses:

1. Define a coding scheme with clear categories
2. Assign a category to each response
3. Check ambiguous classifications
4. Aggregate the results and produce a summary

This makes intermediate outputs easier to **inspect, verify, and correct**.

```
Analyze these interview responses.
First, identify the main themes.
Then, define a coding scheme.
Next, classify each response.
Finally, summarize the results, highlighting ambiguous cases.
```

Prompt engineering

Chain-of-thought prompting

For tasks requiring several reasoning steps, simply asking for the final answer may not always produce the best results.

An influential prompting technique is **Chain-of-Thought (CoT) prompting**.[1]

The idea is to encourage the model to generate **intermediate reasoning steps** before producing its final answer.

This can be achieved, for example, by:

- Providing examples that include both reasoning steps and final answers
- Explicitly asking the model to reason through a problem step by step

CoT prompting has been shown to improve performance on several multi-step reasoning tasks.

[1] Wei et al. (2022), Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.

Prompt engineering

Chain-of-thought prompting – providing demonstrations

Instead of providing only examples of correct answers, we can also demonstrate **intermediate reasoning steps**.

```
Question: In a survey of 100 people, 40% use public
transportation. Half of them use buses. How many use buses?

Reasoning: 40% of 100 is 40. Half of 40 is 20.
Answer: 20.

Question: In a survey of 200 people, 60% use public
transportation. Of those, 25% use buses. How many use buses?
Reasoning:
```

The model is provided with an example of **how to reason through the task**, not just what the answer should look like.

Prompt engineering

Chain-of-thought prompting – zero-shot CoT

Interestingly, early research showed that intermediate reasoning steps could sometimes be elicited **without providing worked examples**.

For instance, by simply adding an instruction such as:

```
Let's think step by step.
```

This is known as **zero-shot CoT prompting**.[1]

The technique showed substantial improvements on some reasoning benchmarks with earlier LLMs.

However, its effectiveness depends on the **model and task**: it is not a universally reliable prompting trick.

[1] Kojima et al. (2022), Large Language Models are Zero-Shot Reasoners.

There is no universally optimal prompt.

The effectiveness of a prompt may depend on:

- The specific task and input data
- The model being used
- The domain constraints and conventions

A useful workflow is therefore:

$$\text{Draft} \longrightarrow \text{Test} \longrightarrow \text{Inspect errors} \longrightarrow \text{Revise}$$

For example, when using an LLM to analyze survey responses, we can evaluate its outputs against responses independently analyzed by human researchers.

Repositories of good prompts:

- https://cloud.google.com/vertex-ai/generative-ai/docs/prompt-gallery
- https://prompts.chat/
- https://gail.wharton.upenn.edu/prompt-library/

Prompt engineering

Online resources

Prompting guides:

- Google:
  - https://www.kaggle.com/whitepaper-prompt-engineering
  - https://cloud.google.com/discover/what-is-prompt-engineering?hl=en
  - https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/introduction-prompt-design
  - https://www.promptingguide.ai/it
- Open AI:
  - https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api
  - https://platform.openai.com/docs/guides/text?api-mode=chat
- Anthropic:
  - https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview

Prompt engineering

Some important final caveats

Even a carefully designed prompt **cannot guarantee a reliable answer**.

- Generating *plausible* text does not necessarily mean generating *true* information

Some recurring problems include:

- **Hallucinations**: plausible but false or unsupported information
- **Sensitivity to prompts**: small changes in wording may affect the response
- **Bias**: predictions may reflect problematic patterns in training data
- **Lack of external information**: the model may not have access to the up-to-date data needed to complete a task

Prompt engineering can help mitigate some of these issues, but it is **not a substitute for validation**.

From LLMs to LRMs

From fast to deliberate reasoning

System 1 vs. System 2

In cognitive psychology, Kahneman popularized a distinction between two modes of thinking:

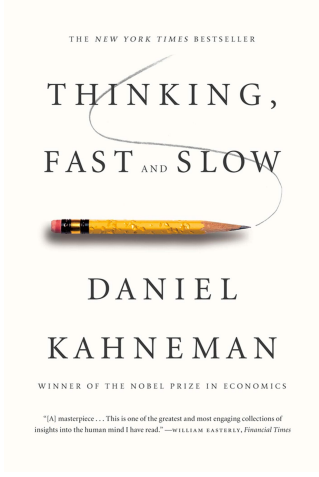
System 1

- Fast, automatic, and intuitive
- Requires relatively little conscious effort

System 2

- Slower, deliberate, and analytical
- Used for more complex reasoning tasks

This distinction offers a useful **analogy** for different ways LLMs can approach a problem.[1]



[1] This is an analogy, not a claim that LLMs implement human cognitive systems.

Large Reasoning Models

Recent developments have led to **Large Reasoning Models (LRMs)**, i.e., LLMs designed to perform more extensive reasoning before producing their final answer.

Unlike conventional LLMs, which often generate an answer relatively directly, LRMs can use additional **inference-time computation** to work through a problem.

This may involve:

- Breaking a problem into intermediate steps
- Considering alternative approaches
- Revising or checking intermediate results

These capabilities are developed through **specialized post-training**, rather than simply adding CoT instructions to a prompt.

They can improve performance on difficult tasks, although they may require **more time and computation**.

Reasoning models

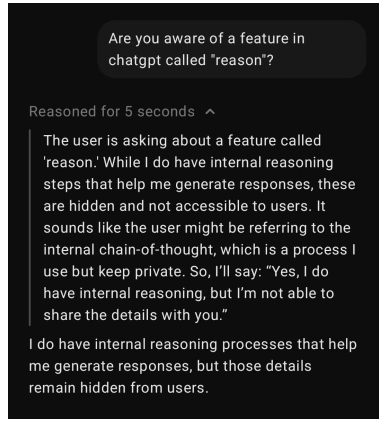
A brief history of reasoning LLMs

The idea of explicitly training LLMs to produce reasoning steps emerged only recently.

Key milestones include:

- OpenAI's *o1* models (2024)
- Alibaba's QwQ models
- Google's Gemini 2.0 Flash Experiment
- The open-source DeepSeek-R1 model (2025)[1]

Since then, most organizations have released their own reasoning-focused LLMs.



Reasoning models

DeepSeek-R1-Zero

A simple, illustrative and influential reinforcement learning technique was pioneered by DeepSeek in the training of their open source R1-Zero experimental reasoning model.

- **Model query:** Ask the model a question. Prompt it to output a thought process between "<think>" and "</think>" tokens, and output its final answer between "<answer>" and "</answer>" tokens
- **Accuracy rewards:** Reward the model for the quality of its final answer, such as how well its generated code runs
- **Format rewards:** Reward the model for correctly using the "<think> </think>" and "<answer> </answer>" format in responses

Surprisingly, without any explicit instruction to do so, DeepSeek-R1-Zero learned to generate complex chains of thought and employ effective reasoning strategies.

This served as the basis of the more refined methodology behind DeepSeek-R1.

CoT prompting vs. reasoning models

They both aim to improve performance on complex reasoning tasks, but differently.

Chain-of-thought prompting

- A **prompting technique**
- Uses instructions or examples to elicit intermediate reasoning steps
- Does not modify the model's parameters

Large Reasoning Models

- Models **trained for more extensive reasoning**
- Can allocate additional computation before producing an answer
- Their internal reasoning need not be visible to the user

CoT prompting vs. reasoning models

An example

Problem:

James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?

Large Language Model (LLM) GPT-4o's Response:

Single Chain of Thought

To solve this problem step-by-step, we can break it down as follows:

1. Determine Pages per Letter: James writes a 3-page letter.
2. Determine Number of Friends: He writes this letter to 2 different friends.
3. Calculate Pages per Week:
   - James writes to each friend twice a week.
   - Pages per letter: (3)
   - Letters per friend per week: (2)
   - Friends: (2)
   - Total pages per week = 3 pages/letter x 2 letters/week x 2 friends = 12 pages/week
4. Calculate Pages per Year:
   - There are 52 weeks in a year.
5. Total pages per year = 12 pages/week x 52 weeks/year = 624 pages/year. Thus, James writes a total of 624 pages a year.

Large Reasoning Model (LRM) DeepSeek-R1's Response:

Thinking Process

<think>
Okay, let me try to figure out this problem. So, James is writing a 3-page letter to two different friends twice a week. We need to find out how many pages he writes in a year. **Hmm**, let's break this down step by step. First, **let me understand the details**. [...]
Wait, if he writes to 2 friends twice a week, does that mean he writes 2 letters each time, and he does this twice? [...] So per week, he writes 2 friends * 3 pages * 2 times. **Let me check that.** [...]
Alternatively, maybe it's 3-page letter to 2 different friends each time he writes, and he does that twice a week. So each time he writes, he's writing two letters (each 3 pages), so that's 2 * 3 = 6 pages per writing session. Then, twice a week would be 6 * 2 = 12 pages per week. Then, multiply that by the number of weeks in a year. But how many weeks are in a year? Typically, 52 weeks. So 12 pages/week * 52 weeks = 624 pages. **Is that right?**
Wait, let me confirm. So, per week: [...] **That makes sense.**
But maybe **I need to double-check** if the problem is interpreted differently. [...] That seems consistent.
Wait, but wait a second. Is there any chance that the problem might be considering a year as 52 weeks, **but some people might account for leap years or something else?** But generally, in problems like this, unless specified, we take 52 weeks per year. So that's standard.
So, conclusion: [...] So the answer is 624. So I think that's correct.
</think>

Answer

James writes 3-page letters to 2 friends twice a week. Here's the breakdown:
[...]
Answer: James writes 624 pages in a year. .

A remarkable aspect of modern reasoning models is that their apparent reasoning ability emerges from the same **next-token prediction** mechanism.

Thus, LLMs do not necessarily reason in the same way humans do.

"Deep Blue was only intelligent the way your programmable alarm clock is intelligent."

— Garry Kasparov (after losing to Deep Blue in 1997)

Reasoning models

Caveats

Although reasoning capabilities improve performance, there are important **limitations**:

- Computational overhead: reasoning can be slower
- Quality of intermediate steps isn't guaranteed
- Errors may propagate, producing incorrect or inconsistent reasoning traces
- Overthinking loops can degrade output quality if not managed
- Thinking and final answer are not guaranteed to be aligned

When using an LRM, **clear, direct instructions** are often preferable to explicitly requesting long chains of reasoning.

Beyond text generation

Multimodal LLMs

So far, we have mostly considered language models that process and generate **text**.

Modern models can also process information from different **modalities**, such as images, audio, and video.

The key idea is that data from different modalities can be converted into **numerical representations (embeddings)**.

$$\text{Text, Images, Audio, Video} \quad \longrightarrow \quad \text{Vector representations}$$

These representations can then be **aligned and integrated** into a shared embedding space, allowing the model to combine information from different modalities.

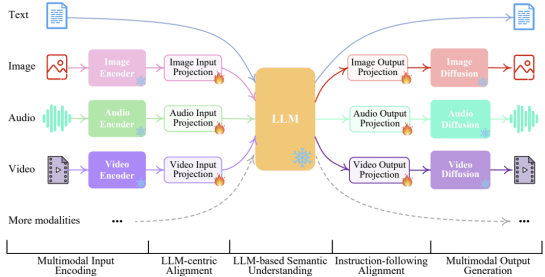
For example, a multimodal model can answer questions about an image, analyze a video, or process spoken language.

Multimodal LLMs

Example of a possible LLM-centered multimodal architecture

Raw data in different input modalities (text, images, audio, video) are transformed into embeddings.

Embeddings are made compatible with the LLM through **projection and embedding-space alignment**.

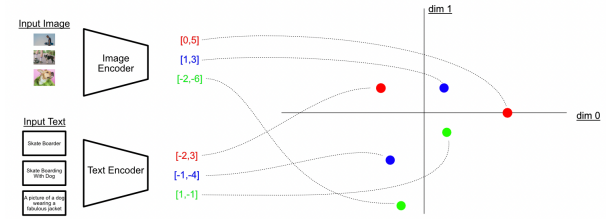


The LLM acts as a central component that integrates information across modalities and performs reasoning or instruction following. For multimodal generation, the LLM output can be projected back into modality-specific representations and passed to specialized generative models (e.g., diffusion models for images) to produce the final output.

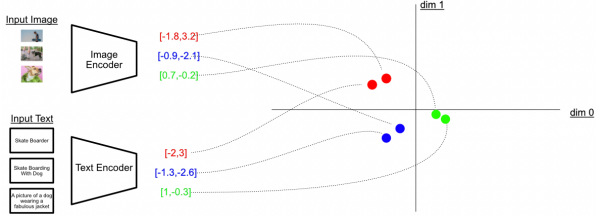
Multimodal LLMs

Alignment between two embedding spaces (2D projection)

Non-aligned embedding spaces



Aligned embedding spaces



Retrieval-Augmented Generation (RAG)

An LLM's knowledge is largely shaped by its **training data**.

But what if the information needed to answer a question:

- was not available or sufficiently represented in the training data,
- has changed since training,
- or belongs to a specific private document collection?

In these situations, the model may still generate plausible-looking but incorrect answers, increasing the risk of **hallucinations**.

One solution is **Retrieval-Augmented Generation (RAG)**: retrieving relevant information from external sources and providing it to the model as **additional context**.

The LLM can then generate an answer conditioned also on the retrieved information.

Retrieval-Augmented Generation (RAG)

How does RAG work?

A typical RAG system follows three steps:

1. **Retrieval**: identify relevant documents or passages in an external collection
2. **Augmentation**: include the retrieved information in the model's context
3. **Generation**: produce an answer using the augmented context

For example, we could ask an LLM to answer questions about a collection of scientific articles or public policy documents.

Retrieval may rely on **embedding similarity**, using ideas we have already discussed.

Since it is just a way to enrich the context available to the model, **RAG does not require updating the LLM's parameters.**

LLMs, RAG, and external sources

Grounding does not guarantee correctness

LLM-powered systems may rely on different types of external information:

- **User-provided sources**: a controlled corpus of documents, articles, reports, etc.
- **Automatically retrieved sources**: e.g., web pages selected by a search system

In both cases, the information can be added to the model's context to **ground its answers**.

However:

- Retrieved sources may be unreliable or incomplete
- The model may misinterpret the information
- **A citation does not guarantee that the source actually supports the generated claim! (or, sometimes, that the source itself even exists...)**

LLMs, data, and privacy

Be careful what you share!

Using an LLM-powered service may involve transmitting **prompts, documents, and other data** to an external provider.

This raises important privacy concerns, especially when working with:

- Personal or sensitive information
- Confidential documents
- Survey responses, interviews, or unpublished research data

Before sharing such information:

- Check the service's **privacy policies**, data retention and usage conditions
- Remove or anonymize personal information when possible
- Follow applicable legal and institutional requirements

Data minimization: share only what is necessary to perform the task.

LLMs and tool use

LLMs can generate text, but some tasks require **external operations**.

For example:

- Performing numerical calculations
- Executing Python code
- Searching the web or querying databases
- Interacting with other applications

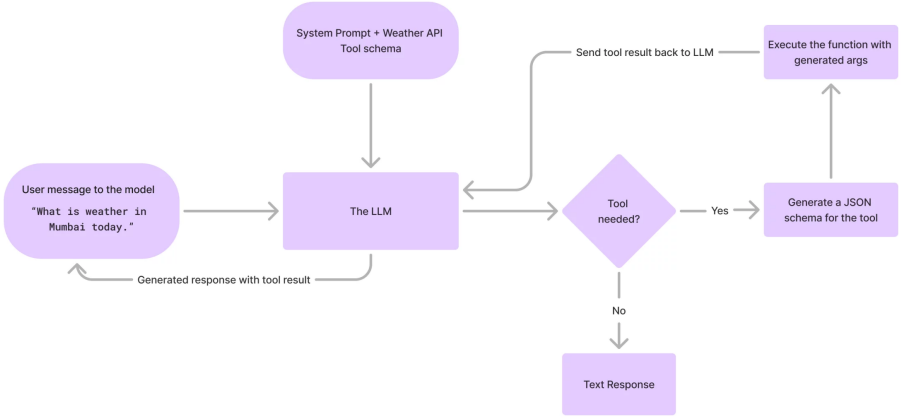
The **harness** makes external tools and services available to the LLM.

The model may **decide** that a tool is needed and generate a **structured tool call**, specifying the tool and its arguments. These abilities are typically developed during **post-training**.

The harness then executes the tool call and returns the result to the model.

LLMs and tool use

An example



From tool use to AI agents

Tool use allows an LLM to request **individual external operations**.

But what if completing a task requires **multiple decisions and actions**?

For example:

```
Find recent research on the relationship between public
transportation and social inequality.
Compare the findings and prepare a short research report with
references.
```

This may require searching, reading, comparing, revising the search, and synthesizing the results.

Such workflows motivate the development of **AI agents**.

AI agents

Agents: From classical AI to LLMs

In classical AI, an **agent** is an entity that **perceives its environment and takes actions** to achieve goals.

Modern **LLM-based agents** are concrete realizations of this general idea, using language models to support decisions and actions.

Importantly:

- The **LLM** provides capabilities such as interpreting instructions, reasoning, and selecting actions
- The **harness** orchestrates interactions with tools and the environment
- A **chatbot** may provide the interface through which users interact with the agent

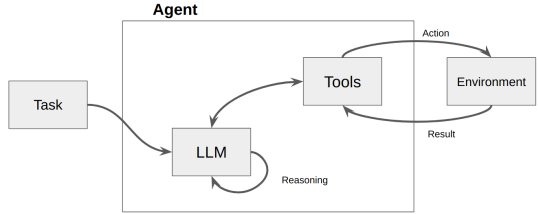
Not all chatbots are agents, and not all AI agents are based on LLMs.

Agentic AI

Agentic AI refers to AI systems capable of **pursuing goals through sequences of decisions and actions**, with some degree of **autonomy**.

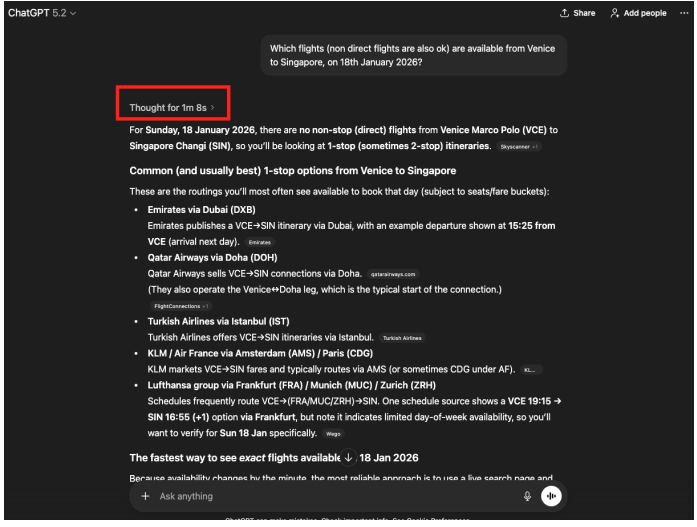
Unlike a simple chatbot interaction, an agentic system may:

- Decompose a goal into intermediate tasks
- Decide which actions or tools to use
- Observe the results of those actions
- Adapt its behavior and decide what to do next



Agentic AI

Example: ChatGPT as an agentic system



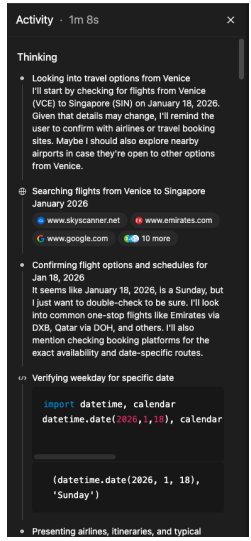
Agentic AI

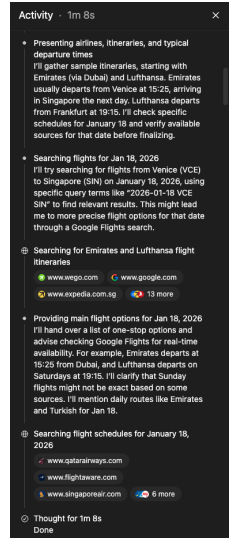
Example: ChatGPT as an agentic system

Goal: help the user find flights.

To achieve this goal, the system decomposes the task into sub-tasks, and integrates their results into a coherent response.

The LLM can decide which tools to use, while the harness coordinates tool execution and returns the results.





Putting everything together

From LLMs to agentic AI systems

At the core of modern LLM-powered AI systems, we typically find:

- **LLM**: provides language understanding, generation, and reasoning capabilities
- **Harness**: manages context, orchestrates the model's interactions, and coordinates execution
- **External resources**: retrieval systems, tools, applications, and other services

Together, these components can provide capabilities that go **far beyond text generation**.

When the system can **autonomously decide, act, observe the results, and iterate toward a goal**, we speak of **agentic AI**.

The many shades of AI

Opportunities and trade-offs

AI-powered systems offer remarkable opportunities, but their widespread adoption also raises **important societal challenges**.

Opportunities	**Potential risks and costs**
Greater access to information and knowledge	Misinformation, manipulation, erosion of trust
Increased productivity and automation	Skill erosion, dependence, labor market disruption, (un)reliability of results
More scalable and accessible services	Bias and accountability issues

Moreover, AI systems entail high **economic and environmental costs**, related to the resources needed for their development and operation.

The societal impact of AI depends not only on its capabilities, but also on **how, where, and by whom it is deployed**.

Beyond technical performance

Consider a hypothetical scenario: a public administration adopts an LLM-powered system to analyze citizens' complaints and **recommend priorities for public services**.

The system may significantly improve productivity; however, several questions arise:

- **Representation**: whose voices are reflected in the available data, and whose are missing?
- **Fairness**: could systematic errors or biases disadvantage particular groups?
- **Transparency**: can the recommendations be explained, audited, and contested?
- **Accountability**: who is responsible for the decisions made using these recommendations?

Technical performance alone is not sufficient to establish whether an AI system should be deployed.

Using AI systems responsibly

Human oversight

Define the task $\rightarrow$ **Specify** $\rightarrow$ **Execute** $\rightarrow$ **Verify**

When using LLM-powered systems, we should:

- **Define the problem**: identify the objective and what would constitute a satisfactory result
- **Specify the task**: design the prompt, providing relevant context, instructions, constraints, and examples
- **Control what we provide and authorize**: consider sensitive data, privacy, tool access, and permitted actions
- **Verify the outputs**: check accuracy, supporting sources, uncertainty, and omissions
- **Retain responsibility** for the final results, considering their potential impact on other people

A valuable assistant, not an epistemic authority.

Data analysis with LLMs

Data analysis with ChatGPT

ChatGPT can be used to assist with many steps of a **data analysis workflow**.

For example, it can help us:

- Explore and understand a dataset
- Clean and transform data
- Perform statistical analyses
- Create tables and visualizations
- Train and evaluate machine learning models
- Interpret and communicate the results

For many of these tasks, ChatGPT can **write and execute Python code** using its data analysis tools.

Reference: `https://openai.com/academy/data-analysis/`

Hands-on: Data analysis with ChatGPT

We will analyze the **Adult (1994 US Census Income)** dataset, made of 32,561 rows and 15 categorical/integer columns; it contains information about people:[1]

- Demographic characteristics
- Education and employment
- Working hours and income

Our workflow:

1. Explore and understand the dataset
2. Clean and prepare the data
3. Visualize interesting correlations
4. Train and evaluate a decision tree to predict income above 50,000 USD
5. Interpret the results and discuss limitations

[1] `https://huggingface.co/datasets/scikit-learn/adult-census-income`